

Resumen — Integración de Spring Security (AsistenciaFGK / Oportunidades)

Fecha: 03-may-2026

Proyecto: Sistema de asistencia FGK (módulo `Oportunidades`)

Requisito evaluado: Integrar seguridad (control de acceso y protección de rutas en base a roles de usuarios) con Spring Security.*

1) ¿Qué se implementó?

Se integró **Spring Security** en una aplicación **Spring Boot MVC + Thymeleaf**, con:

- **Autenticación** por formulario (`/login`) usando credenciales almacenadas en base de datos.
- **Autorización por roles** (protección de rutas) usando:
 - reglas en `SecurityFilterChain` (por patrón de URL), y
 - anotaciones `@PreAuthorize` en controllers.
- **Gestión de contraseñas** con hash **BCrypt** (`PasswordEncoder`).
- **Cierre de sesión** (logout), invalidación de sesión y eliminación de cookie `JSESSIONID`.

2) Componentes de seguridad (cómo funcionan)

2.1 Autenticación (login)

- La página de login es **personalizada** y se sirve desde el controller.
- El POST a `/login` lo maneja Spring Security.
- La verificación de credenciales se realiza con un `UserDetailsService` propio que:
 - 1) busca el usuario en la BD por `username`,
 - 2) valida que el usuario esté activo, y
 - 3) convierte el rol de la entidad a `GrantedAuthority`.

Resultado: Spring Security puede autenticar usuarios reales del sistema.

2.2 Autorización (roles + rutas protegidas)

Se definió control de acceso por roles de dos formas complementarias:

1) **Por patrones de ruta** en la configuración:

- rutas públicas (`permitAll`) como `/login` y recursos estáticos;
- rutas con restricciones (`hasRole`) como `/admin/**`, `/docente/**`, `/supervisor/**`;
- cualquier otra ruta requiere autenticación (`anyRequest().authenticated()`).

2) **Por anotaciones en controllers**:

- `@PreAuthorize("hasRole('ADMIN')")` en controllers administrativos, etc.

Esto demuestra control de acceso a nivel de configuración y a nivel de capa web.

2.3 Redirección según rol

Luego de autenticarse, se redirige a una pantalla distinta según el rol del usuario (ADMIN/DOCENTE/SUPERVISOR u otro caso general).

2.4 Logout

- Se configuró una URL de logout y redirección de salida al login.
- Se invalida la sesión y se elimina la cookie de sesión.

2.5 CSRF

En la configuración actual se **deshabilita CSRF**.

- Ventaja: simplifica formularios/acciones en desarrollo.
- Consideración: en aplicaciones web con formularios (Thymeleaf), normalmente se recomienda habilitar CSRF y usar el token en los forms.

3) Mapa de roles y rutas

Reglas principales (según la configuración):

Ruta (pattern)	Acceso	Rol requerido
<code>/login</code> , <code>/css/**</code> , <code>/js/**</code> , <code>/images/**</code> , <code>/usuarios</code>	Público	—
<code>/asistencia/registrar</code>	Público	—
<code>/docente/**</code>	Protegido	<code>DOCENTE</code>
<code>/supervisor/**</code>	Protegido	<code>SUPERVISOR</code>
<code>/admin/**</code>	Protegido	<code>ADMIN</code>
Cualquier otra	Protegido	Autenticado

Roles manejados en el sistema: `ROLE_ADMIN`, `ROLE_DOCENTE`, `ROLE_SUPERVISOR`, `ROLE_ALUMNO`.

4) Evidencia en el código (archivos clave)

- Configuración de seguridad (reglas de rutas, login, logout, roles):
 - `src/main/java/AsistenciaFGK/Oportunidades/security/SecurityConfig.java`
- Servicio de autenticación (carga de usuario desde BD + authorities):
 - `src/main/java/AsistenciaFGK/Oportunidades/security/CustomUserDetailsService.java`
- Modelo de usuarios y rol:
 - `src/main/java/AsistenciaFGK/Oportunidades/models/Usuario.java`
 - `src/main/java/AsistenciaFGK/Oportunidades/models/Role.java`
- Controllers protegidos por rol:
 - `src/main/java/AsistenciaFGK/Oportunidades/controllers/AdminController.java`
 - `src/main/java/AsistenciaFGK/Oportunidades/controllers/SeccionController.java`
 - `src/main/java/AsistenciaFGK/Oportunidades/controllers/AlumnoController.java`
 - `src/main/java/AsistenciaFGK/Oportunidades/controllers/SupervisorController.java`
 - `src/main/java/AsistenciaFGK/Oportunidades/controllers/DocenteController.java`
- Vista de login (Thymeleaf):
 - `src/main/resources/templates/login.html`

5) Cómo demostrarlo rápido (checklist)

- 1) Ir a `/login` → debe abrir el formulario.
- 2) Intentar entrar a `/admin/usuarios` sin loguearse → debe pedir autenticación.
- 3) Loguearse con un usuario ****ADMIN**** → debe redirigir a zona admin.
- 4) Loguearse con ****DOCENTE**** → debe redirigir a `/docente`.
- 5) Loguearse con ****SUPERVISOR**** → debe redirigir a `/supervisor`.
- 6) Usar logout → debe cerrar sesión y volver a `/login?logout=true`.

6) Conclusión

El proyecto **sí integra Spring Security** y cumple el requisito evaluado de **proteger rutas por roles** y **controlar acceso** en un sistema MVC con usuarios persistentes.